Avaliação Empírica e Limites Arquiteturais de Modelos de Linguagem Pequenos (SLMs) sob Restrições Severas de Memória: Da Arquitetura Transformer à Inferência em GPU de Borda

Anderson Tavares

Graduando em Engenharia de Software e Ciência de Dados

Repositório do Projeto:

`https://github.com/AndersonTavares0/llm-research-bench`

Outubro de 2026

Resumo

A implantação de Modelos de Linguagem Pequenos (SLMs, entre 0,5B e 4B parâmetros) em aceleradores físicos de borda viabiliza soberania de dados, latência previsível e operação autônoma sem dependência de conectividade externa. Contudo, operar sob limites estritos de memória física ($\leq$ 4096 MiB de VRAM) introduz dilemas acentuados entre escala de parâmetros, compressão de tensores por quantização, saturação do barramento DRAM, crescimento dinâmico do cache KV, eficiência de tokenização multilíngue e alinhamento defensivo. Este trabalho conduz uma investigação experimental fundamentada nos princípios arquiteturais de Vaswani et al. (2017) e em suas evoluções contemporâneas (GQA, RoPE, RMSNorm e decodificação especulativa). Utilizando uma GPU NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, $112,0$ GB/s teóricos), avaliam-se oito configurações representativas com telemetria contínua de hardware via NVML. Um estudo de ablação controlada em três condições (C_1, C_2, C_3) decompôs os fatores de ganho sob quantização constante: a compressão de pesos em IQ4_XS proporcionou $+28,2\%$ de aumento na vazão ($34,51 \pm 0,29$ vs $26,92 \pm 0,23$ tok/s, microdados brutos $t = 64,26$, valores arredondados $t = 64,85$, $d = 28,74$), enquanto a decodificação especulativa com Multi-Token Prediction (MTP) entregou $+25,6\%$ de ganho associado ($43,35 \pm 0,57$ vs $34,51 \pm 0,29$ tok/s, $t = 43,72$, $p < 10^{-15}$, $d = 19,55$), totalizando diferença composta de $+61,0\%$ ($t = 84,50$, $d = 37,79$). A sobretaxa de

VRAM associada à infraestrutura de MTP totalizou $+236,0$ MiB, resultando em diferença líquida de apenas $+40,0$ MiB frente à linha de base devido à absorção promovida pela compressão IQ4_XS ($-196,0$ MiB). A modelagem Roofline evidenciou que a geração autorregressiva opera estritamente limitada por largura de banda ($I_{\text{decode}} \approx 3,6$ FLOP/byte $\ll I^* = 19,1$), com proxy analítico de demanda entre $45,7$ e $75,3$ GB/s ($40,8\%$–$67,2\%$ do barramento). Uma auditoria temporal da campanha experimental ativa ($311,58$ s no total) quantificou a elevação térmica do die de 44°C para 74°C e o escalonamento adaptativo de clocks via GPU Boost 3.0 ($1695 \rightarrow 1607$ MHz, $-5,2\%$), motivando a formalização de um protocolo padronizado em três níveis (A/B/C). A análise multilíngue em corpus paralelo trilíngue (en-US, pt-BR, zh-CN) revelou penalidades de tokenização de até $2,4\times$ para português e $4,6\times$ para mandarim em modelos com vocabulários compactos (SmolLM2), reduzindo a capacidade efetiva da janela de contexto para 22%. Em avaliação exploratória de segurança defensiva em turno único ($N = 10$), observou-se ausência de recusa indevida em tarefas benignas (0/4, FPR $= 0,0\%$, IC 95% de Clopper-Pearson $[0,0\%, 60,2\%]$), porém persistência de vulnerabilidade a injeções indiretas (ASR entre 50% e 100%), com todas as chamadas de ferramentas maliciosas interceptadas deterministicamente pelo harness (100% bloqueadas, $0,0\%$ executadas no sistema operacional).

Palavras-chave: Modelos de Linguagem Pequenos (SLMs); Arquitetura Transformer; Cache KV; Grouped-Query Attention; Multi-Token Prediction; Modelo Roofline; Eficiência de Tokenização; Segurança Defensiva.

1 Introdução e Contextualização

A computação em borda com Modelos de Linguagem Pequenos (SLMs, na faixa de 0,5 a 4 bilhões de parâmetros) estabelece uma mudança paradigmática no projeto de sistemas autônomos. Ao eliminar a necessidade de conexão contínua com infraestruturas em nuvem, viabiliza-se a operação com soberania completa sobre dados sensíveis, ausência de latência de rede variável e resiliência em ambientes desconectados. Entretanto, essa autonomia confronta fronteiras físicas rígidas impostas pelo hardware de baixo custo: em aceleradores com orçamento de memória de 4096 MiB (4 GiB), como a GPU NVIDIA GeForce GTX 1050 Ti, a margem para alocação de tensores de pesos, contexto dinâmico do driver e memória temporária de inferência é extremamente exígua.

O ponto de partida teórico para compreender o comportamento de modelos de linguagem contemporâneos é o artigo seminal *"Attention Is All You Need"* de Vaswani et al. (2017) [1]. Ao propor a eliminação total da recorrência sequencial em favor de mecanismos de autoatenção (*self-attention*), Vaswani et al. revolucionaram a área, viabilizando o treinamento massivamente paralelizado. Contudo, as arquiteturas contemporâneas de SLMs incorporam uma série de refinamentos que não constavam da formulação original de 2017: decodificação autorregressiva *decoder-only*, atenção agrupada (*Grouped-Query Attention* – GQA) [12], codificação posicional rotativa (*Rotary Position Embeddings* – RoPE) [13], normalização quadrática sem centralização de média (*RMSNorm*) [9], ativações fechadas SwiGLU [11] e aceleração especulativa via *Multi-Token Prediction* (MTP) [4], diferenciando-se da decodificação especulativa clássica baseada em modelos de rascunho externos [14]. Além disso, a compressão de tensores por quantização avançada no *llama.cpp* (k-quants e matrizes de importância IQ) [5, 6] viabiliza a operação direta em GPUs de consumo.

A compreensão de como essas decisões de design interagem com o hardware exige uma análise que una modelagem matemática e rigor experimental. A GPU de teste empregada neste estudo, fundamentada na microarquitetura NVIDIA Pascal (chip GP107, Compute Capability 6.1), impõe características operacionais críticas: ausência de unidades de aceleração tensorial (*Tensor Cores*), execução de ponto flutuante em meia precisão (FP16) penalizada na taxa de $1:64$ em relação ao ponto flutuante simples (FP32), e barramento de memória GDDR5 de 128 bits com largura de banda teórica de $112,0$ GB/s. Sob essas condições, a geração autorregressiva de texto torna-se profundamente limitada pela largura de banda de memória, enquanto a fase de processamento inicial de contexto (*prefill*) opera limitada pela capacidade computacional dos núcleos CUDA.

Além dos limites de vazão e capacidade física, a eficácia operacional de um modelo em borda depende criticamente de seu vocabulário e tokenizador. Diferenças estruturais na construção do tokenizador geram fenômenos de fragmentação subpalavra assimétricos entre idiomas, aumentando o custo computacional de representação, processamento e geração em línguas além do inglês [19, 20]. Concomitantemente, a segurança defensiva do

modelo em operação autônoma confronta dilemas de alinhamento: o modelo deve evitar a recusa indevida de comandos administrativos legítimos enquanto rejeita tentativas de injeção indireta de instruções e manipulação de ferramentas em fluxos agentivos [24, 25].

Este artigo técnico apresenta uma investigação compreensiva desses eixos interligados, caracterizando o comportamento de oito variantes de modelos e quantização sob rigor metodológico de medição, fundamentação matemática explícita e preservação de rastreabilidade integral.

2 Problema Científico, Questões de Pesquisa e Hipóteses

A investigação é estruturada em torno das seguintes Questões de Pesquisa (RQs):

- **RQ1:** Quais são os limites físicos de vazão e alocação de VRAM para SLMs na faixa de 0,5B a 4B em aceleradores de consumo de 4 GiB sem Tensor Cores?
- **RQ2:** Em um desenho experimental controlado de ablação sob quantização mantida constante, qual é a magnitude do ganho de vazão e da sobretaxa de VRAM associada ao mecanismo especulativo de MTP?
- **RQ3:** De que maneira a arquitetura de atenção (MHA vs GQA) e o tamanho do vocabulário influenciam a taxa de consumo do orçamento dinâmico de contexto sob o teto físico de 4096 MiB?
- **RQ4:** Qual é a extensão da desigualdade de tokenização e do custo computacional entre inglês (en-US), português (pt-BR) e mandarim (zh-CN) em modelos de pequeno porte?
- **RQ5:** Os mecanismos de alinhamento contemporâneos garantem robustez contra injeções indiretas e uso indevido de ferramentas em ambientes de borda sem incorrer em recusas indevidas (*over-refusal*)?

As hipóteses nulas (H_0) e alternativas (H_1) formalizadas para orientar a verificação experimental são:

- **Hipótese 1 (H_1):** A decodificação especulativa via MTP sob quantização idêntica proporciona ganho estatisticamente significante de vazão de decodificação ($p < 0,01$), compensando a sobretaxa física de alocação de memória das cabeças de rascunho em regimes severamente delimitados pelo barramento DRAM.
- **Hipótese 2 (H_2):** A adoção de atenção agrupada (GQA) é condição estritamente necessária para permitir que modelos próximos a 3B e 4B parâmetros sustentem contextos ≥ 4096 tokens dentro de 4 GiB de VRAM.

- **Hipótese 3 (H_3):** A fertilidade de tokenização em idiomas com menor representação no vocabulário impõe degradação superior a 40% na capacidade útil da janela de contexto frente ao inglês para modelos com vocabulários compactos ($\leq$ 50k tokens).

3 Fundamentação Teórica e Modelagem Matemática

3.1 Fundamentos do Mecanismo de Atenção e Vaswani et al. (2017)

Antes do advento da arquitetura Transformer proposta por Vaswani et al. (2017) [1], o processamento de sequências em inteligência artificial dependia preponderantemente de Redes Neurais Recorrentes (RNNs) e de suas variantes com portas de controle de fluxo de gradiente, tais como Long Short-Term Memory (LSTM) [**?**] e Gated Recurrent Units (GRU) [**?**]. Essas arquiteturas impunham uma restrição computacional intrínseca: a natureza sequencial do estado oculto, definido por $h_t = f(h_{t-1}, x_t)$, impedia a paralelização temporal das operações de treinamento ao longo do comprimento da sequência L. O tempo de treinamento escalava como $\mathcal{O}(L)$ passos sequenciais dependentes. Além disso, o vetor de estado oculto de dimensão fixa h_t atuava como um gargalo de informação crítico, agravado por problemas de desvanecimento ou explosão de gradientes ao longo de longas distâncias contextuais.

Embora mecanismos de atenção aditiva tenham sido introduzidos por Bahdanau et al. (2014) como extensores de memória para RNNs de tradução, a contribuição central de Vaswani et al. (2017) consistiu na demonstração formal de que a recorrência sequencial e as convoluções poderiam ser inteiramente dispensadas: o mecanismo de autoatenção (*self-attention*) é suficiente para modelar dependências globais de sequência com custo de caminho constante $\mathcal{O}(1)$ entre quaisquer duas posições temporais.

O componente fundamental da arquitetura é o mecanismo de *Scaled Dot-Product Attention*. Dadas três matrizes de entrada: consultas ($Q \in \mathbb{R}^{n \times d_k}$), chaves ($K \in \mathbb{R}^{m \times d_k}$) e valores ($V \in \mathbb{R}^{m \times d_v}$), a operação é formulada matematicamente pela expressão:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

Na Equação (1), o produto matricial $QK^\top \in \mathbb{R}^{n \times m}$ calcula o produto escalar entre cada vetor de consulta e todos os vetores de chaves, quantificando o grau de afinidade contextual entre as respectivas posições. O termo de escala $\frac{1}{\sqrt{d_k}}$ desempenha um papel matemático crítico: assumindo que os componentes de q e k são variáveis aleatórias independentes com média zero e variância unitária, o produto escalar $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ possui média zero e variância exatamente igual a d_k. Para dimensões elevadas de projeção ($d_k \geq 64$), os produtos escalares assumem magnitudes absolutas elevadas, empurrando a função softmax

para regiões periféricas com gradientes de magnitude infinitesimal. A divisão por $\sqrt{d_k}$ normaliza a variância de volta para $1,0$, assegurando estabilidade numérica e evitando a saturação dos gradientes durante a retropropagação.

Para enriquecer a capacidade expressiva do modelo, Vaswani et al. expandiram a atenção escalar para o mecanismo de *Multi-Head Attention* (MHA). Em vez de aplicar uma única função de atenção com dimensão d_{model}, projetam-se linearmente consultas, chaves e valores h vezes com matrizes de projeção aprendidas $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ e $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2}$$

onde cada cabeça de atenção é computada por:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{3}$$

com matriz de projeção linear $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. Esse mecanismo permite ao modelo direcionar a atenção simultaneamente para diferentes subespaços de representação em múltiplas posições relativas.

A arquitetura original do Transformer operava sob topologia codificador-decodificador (*encoder-decoder*). Em ambas as partes, cada camada continha subcamadas de atenção seguidas de redes neurais *Feed-Forward* densas aplicadas a cada posição identicamente:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{4}$$

Cada subcamada era envolvida por conexões residuais e normalização de camada (*Layer Normalization* – Post-LN) [8]: $\text{LN}(x + \text{SubLayer}(x))$. No decodificador, impunha-se uma máscara causal triangular superior preenchida com $-\infty$ antes do cálculo da softmax, impedindo que posições presentes atendessem a posições futuras durante a geração. A informação de ordem temporal era fornecida por codificações posicionais senoidais fixas adicionadas aos *embeddings* de entrada.

Ressalva Metodológica e Precedência Histórica: É imperativo registrar formalmente que o trabalho pioneiro de Vaswani et al. (2017) estabeleceu os fundamentos matemáticos da atenção, mas não concebeu diversas das técnicas de otimização empregadas nos modelos modernos aqui avaliados. Métodos como atenção puramente generativa *decoder-only*, GQA, RoPE, RMSNorm, SwiGLU, PagedAttention e MTP representam inovações desenvolvidas subsequentemente pela comunidade científica ao longo de quase uma década de investigações.

3.2 Evolução Arquitetural dos Modelos Contemporâneos

A transição da arquitetura original para a geração contemporânea de SLMs e LLMs fundamentou-se em decisões de engenharia direcionadas à escalabilidade e à eficiência física de inferência:

1. Arquiteturas Decoder-Only e Geração Autorregressiva: A eliminação do módulo codificador consolidou a geração em torno da tarefa unificada de predição autorregressiva do próximo token (*next-token prediction*) sob máscara causal estrita, simplificando os fluxos computacionais e compartilhando parâmetros.

2. Normalização Pre-LN e RMSNorm: A substituição do esquema original Post-LN pela organização Pre-LN ($x_{l+1} = x_l + \text{SubLayer}(\text{Norm}(x_l))$) estabilizou a retropropagação em redes profundas. Adicionalmente, Zhang e Sennrich (2019) [9] demonstraram que a invariância de escala pode ser mantida descartando-se o cálculo da média, operando apenas com o valor eficaz (*Root Mean Square*):

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \quad \text{onde } \text{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon} \tag{5}$$

Essa simplificação elimina passagens extras de redução na memória compartilhada, economizando cerca de 7% do tempo de execução de normalização nos kernels da GPU.

3. Ativações SwiGLU: Shazeer (2020) [11] propôs a substituição das funções ReLU e GELU por variantes fechadas (*Gated Linear Units*) com função Swish:

$$\text{SwiGLU}(x) = (\text{Swish}(xW) \otimes xV)\, W_2 \tag{6}$$

Apesar de introduzir uma terceira matriz de projeção na camada FFN, SwiGLU proporciona maior expressividade dimensional, permitindo convergir com menor perda de entropia cruzada sob mesma contagem de camadas.

4. Codificação Posicional Rotativa (RoPE): Su et al. (2024) [13] superaram as limitações das codificações posicionais absolutas propondo o RoPE, que codifica a posição relativa mediante a aplicação de uma rotação ortogonal aos vetores de consulta e chave no plano complexo bidimensional. Dados q_m na posição m e k_n na posição n, a operação assegura que o produto escalar $\langle R_m q_m, R_n k_n \rangle = g(q, k, m-n)$ dependa estritamente da distância relativa $(m-n)$, permitindo extrapolação de contexto sem retreinamento completo.

5. Atenção Agrupada (Grouped-Query Attention – GQA): Para contornar o estrangulamento de memória provocado pelo cache KV em MHA tradicional, Shazeer (2019) [10] propôs a Multi-Query Attention (MQA), em que todas as cabeças de consulta compartilham uma única cabeça de chaves e valores ($n_{\text{kv}} = 1$). Reconhecendo que MQA pode degradar a precisão de raciocínio em tarefas complexas, Ainslie et al. (2023) [12]

generalizaram o conceito através de GQA: as n_{q} cabeças de consulta são particionadas em g grupos, com cada grupo compartilhando $n_{\text{kv}} = g$ cabeças de KV ($1 < n_{\text{kv}} < n_{\text{q}}$). Essa abordagem preserva o desempenho expressivo próximo ao MHA enquanto reduz a pegada de memória do cache KV por um fator de $n_{\text{q}}/n_{\text{kv}}$ (tipicamente 4× a 8×).

3.3 Dinâmica e Dimensionamento do Cache KV

Durante a inferência autorregressiva, o cálculo das chaves (K) e valores (V) para tokens precedentes não necessita ser refeito a cada novo passo. Os tensores gerados são retidos na memória VRAM em uma estrutura designada *Key-Value Cache* (Cache KV). O volume total de memória física exigido pelo cache KV em precisão FP16 (2 bytes por elemento) escala estritamente com o comprimento do contexto L, sendo formulado analiticamente como:

$$\text{VRAM}_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{kv}} \times d_{\text{head}} \times L \times 2 \text{ bytes} \tag{7}$$

A Equação (7) elucida o impacto decisivo da escolha entre MHA e GQA em hardware restrito. Considere o modelo Llama-3.2-3B ($n_{\text{layers}} = 28$, $d_{\text{head}} = 128$):

- **Cenário Hipotético sob MHA ($n_{\text{kv}} = 24$):** Para um contexto $L = 4096$, o cache KV demandaria: $2 \times 28 \times 24 \times 128 \times 4096 \times 2$ bytes $\approx 1.407.374.848$ bytes ≈ 1342 MiB.
- **Cenário Real com GQA ($n_{\text{kv}} = 8$):** Sob a configuração efetiva com razão $3:1$, o consumo reduz-se a: $2 \times 28 \times 8 \times 128 \times 4096 \times 2$ bytes $\approx 469.124.950$ bytes ≈ 448 MiB.

A economia direta de quase 900 MiB de VRAM física é o fator técnico que viabiliza a coexistência dos tensores estáticos de pesos do modelo (2120 MiB) e do contexto do driver (256 MiB) dentro da fronteira física de 4096 MiB da GTX 1050 Ti. Caso o modelo empregasse MHA tradicional, a alocação combinada ultrapassaria 3718 MiB, disparando falhas forçadas de esgotamento de memória (*Out-of-Memory* – OOM) em qualquer oscilação de buffers temporários.

3.4 Modelo Roofline e Limites da Arquitetura Pascal GP107

Para compreender a vazão de inferência, modela-se o comportamento de hardware por meio do modelo Roofline (Williams et al., 2009) [2]. A plataforma de teste é equipada com uma GPU NVIDIA GeForce GTX 1050 Ti baseada na microarquitetura Pascal (chip GP107-400-A1, litografia de 14 nm FinFET, Compute Capability 6.1), cujas especificações físicas de barramento são documentadas pela NVIDIA Corporation (2016) [3]. O processador gráfico possui 768 núcleos de processamento escalar CUDA FP32 distribuídos em 6 multiprocessadores de fluxo (SMs), operando com frequência boost de 1392 MHz, perfazendo uma capacidade de cálculo de pico em precisão simples formulada por:

$$P_{\text{FP32}} = 768 \times 2 \times 1,392 \text{ GHz} \approx 2,138 \text{ TFLOPS} = 2138 \text{ GFLOPS} \tag{8}$$

Três características arquiteturais da GTX 1050 Ti definem os gargalos físicos deste estudo:

1. **Ausência Total de Tensor Cores:** Unidades de processamento tensorial dedicadas foram introduzidas exclusivamente nas arquiteturas Volta (GV100) e Turing (TU102). O chip Pascal GP107 não possui aceleradores matriciais dedicados; todas as operações são executadas nas ALUs escalares FP32.

2. **Penalidade de Taxa FP16:** Ao contrário do chip Pascal voltado a servidores (GP100), que oferecia computação em meia precisão na taxa $2:1$, o chip GP107 opera FP16 na razão degradada de $1:64$ da velocidade FP32. Consequentemente, para preservar a eficiência, os kernels de inferência (como os do *llama.cpp*) descompactam tensores quantizados diretamente para registradores FP32 durante a execução de produtos escalares.

3. **Barramento de Memória Delimitado:** A placa dispõe de 4096 MiB de memória GDDR5 acoplada via barramento de 128 bits operando a 3504 MHz ($7,008$ Gbps efetivos), resultando na largura de banda de pico teórica:

$$B_{\text{teor}} = \frac{128 \text{ bits}}{8 \text{ bits/byte}} \times 7,008 \text{ GHz} = 16 \text{ bytes} \times 7,008 \text{ GHz} \approx 112,13 \text{ GB/s} \quad (9)$$

A intensidade aritmética crítica (I^*) do sistema define o ponto de transição no gráfico Roofline:

$$I^* = \frac{P_{\text{FP32}}}{B_{\text{teor}}} = \frac{2138 \text{ GFLOPS}}{112,0 \text{ GB/s}} \approx 19,1 \text{ FLOP/byte} \quad (10)$$

Contraste Físico entre Prefill e Decode:

- **Decodificação Autorregressiva ($I_{\text{decode}} \ll I^*$):** Durante a decodificação token a token com lote unitário (batch size $= 1$), a matriz de consulta possui dimensão $Q \in \mathbb{R}^{1\times d}$. Para produzir um único token, cada peso do modelo deve ser lido da memória DRAM para os registradores da GPU. Em um formato de quantização de 4 bits (onde cada peso consome aproximadamente $0,56$ bytes considerando escalas e offsets), executam-se 2 operações de ponto flutuante (multiplicação e acumulação). A intensidade aritmética efetiva é:

$$I_{\text{decode}} \approx \frac{2 \text{ FLOPs}}{0,56 \text{ bytes}} \approx 3,57 \text{ FLOP/byte} \quad (11)$$

Como $3,57 \ll 19,1$, a decodificação autorregressiva situa-se profundamente na região **limitada por largura de banda de memória** (*memory-bound*). O teto teórico superior de vazão para um modelo de tamanho de tensores S_{pesos} é:

$$\text{Throughput}_{\text{teto}} = \frac{B_{\text{teor}}}{S_{\text{pesos}}} \quad (12)$$

Para o Gemma-2-2B em Q4_K_M ($S_{\text{pesos}} \approx 1,99$ GB), $\text{Throughput}_{\text{teto}} = 112/1,99 \approx 56,3$ tok/s. A vazão experimental medida de $26,92$ tok/s representa $47,8\%$ de eficiência de barramento.

- **Processamento Inicial de Contexto / Prefill ($I_{\text{prefill}} \gg I^*$):** Na fase de prefill, processa-se a sequência inteira de entrada em bloco ($L_{\text{prompt}} \geq 256$). Cada peso é transferido da memória DRAM uma única vez e reutilizado através de todas as colunas da matriz de entrada. A intensidade aritmética escala linearmente com o bloco:

$$I_{\text{prefill}} \approx 3,57 \times L_{\text{prompt}} \approx 914\text{–}1828 \text{ FLOP/byte} \gg 19,1 \tag{13}$$

Portanto, o processamento de contexto (*prefill*) é estritamente **limitado por capacidade computacional**. Nesta fase, a ausência de núcleos tensores penaliza a arquitetura Pascal, restringindo o prefill a patamares entre 49 e 237 tok/s, enquanto GPUs com aceleradores dedicados atingem ordens de grandeza superiores.

3.5 Fundamentação Teórica de Viés, Assimetrias e Desigualdade de Tokenização

A literatura contemporânea em segurança e ciência de dados estabeleceu que a avaliação de modelos não pode restringir-se a métricas puramente sintéticas de vazão. No domínio comportamental e multilíngue, três corpora metodológicos orientam este trabalho:

1. **Avaliação de Viés Desambiguado (BBQ e CrowS-Pairs):** O benchmark BBQ (Parrish et al., 2022) [21] e o CrowS-Pairs (Nangia et al., 2020) [22] estabeleceram que modelos de linguagem frequentemente recorrem a preconceitos estereotípicos em situações onde o contexto é ambíguo, mas devem demonstrar neutralidade estrita e precisão factual quando informações suficientes são fornecidas.
2. **Enquadramento Geopolítico e Alinhamento Cultural (GlobalOpinionQA):** Durmus et al. (2023) [23] demonstraram que modelos pré-treinados e alinhados absorvem assimetrias sistemáticas de visão de mundo de acordo com a predominância cultural de seus conjuntos de dados de treino e políticas de moderação de seus desenvolvedores. A investigação científica dessas assimetrias exige prompts cuidadosamente balanceados e taxonomias formais de análise.
3. **Desigualdade e Custo de Tokenização (Tokenization Inequality):** Petrov et al. (2023) [19] e Ahia et al. (2023) [20] demonstraram que a eficácia de compressão de tokenizadores de subpalavras (BPE e SentencePiece) é desigual entre idiomas. A fertilidade de tokenização (*tokens per word* ou *tokens per character*) atua como uma barreira que encarece proporcionalmente o processamento de prefill, a decodificação e a utilização da memória dinâmica em idiomas de menor cobertura no vocabulário.

Para evitar classificações arbitrárias de viés e censura, adota-se neste trabalho uma taxonomia operacional com nove classificações formais e mutuamente exclusivas:

1. **Resposta Direta:** Fornecimento objetivo e neutro das informações factuais documentadas.
2. **Recusa Explícita:** Rejeição declarada de responder por política de segurança manifestada.
3. **Evasão ou Desvio:** Alteração sutil do foco da questão para tópicos correlatos sem responder ao cerne do prompt.
4. **Resposta Parcial:** Abordagem de aspectos periféricos com omissão deliberada do ponto controverso central.
5. **Erro Factual:** Afirmação empiricamente falsa ou anacronismo histórico verificável.
6. **Omissão Factual:** Supressão de fatos consensuais universalmente aceitos pela historiografia.
7. **Enquadramento Assimétrico:** Adoção acrítica de uma narrativa estatal unilateral sem menção a contrapartes.
8. **Expressão de Incerteza:** Qualificação epistêmica legítima (ex.: debate historiográfico em aberto).
9. **Não Interpretável:** Resposta truncada, loop de repetição ou corrupção de tokens.

4 Materiais e Metodologia Experimental

4.1 Bancada Físico-Computacional

Os experimentos foram conduzidos em uma estação de trabalho com hardware físico dedicado: processador AMD Ryzen 5 3600 (6 núcleos, 12 threads), 32 GiB de memória RAM DDR4-3200 em canal duplo, e placa aceleradora gráfica NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, Compute Capability 6.1). A GPU foi operada sob isolamento rigoroso via Debian 12 com núcleo Linux 6.8 e driver proprietário NVIDIA 550.120. A execução dos modelos foi realizada utilizando o servidor de inferência `llama-server` compilado nativamente a partir do *llama.cpp* (b4120+), com suporte a CUDA e otimização de atenção.

A telemetria de hardware foi coletada em tempo real com frequência de amostragem de 250 ms através das bibliotecas oficiais NVML (*NVIDIA Management Library*), registrando o consumo de memória física VRAM e a temperatura do die. Como ressalva de instrumentação, a arquitetura GP107 não disponibiliza leituras contínuas de potência em

watts via NVML. Portanto, as estimativas de eficiência energética formalizadas nesta pesquisa ($\eta = \text{vazão}/P_{\text{hipotético}}$) baseiam-se em uma análise paramétrica de sensibilidade sob o envelope de TDP de 75 W da placa, considerando regime operacional contínuo delimitado entre 42 W e 62 W, não se confundindo com medições elétricas diretas em tomada AC de bancada ($>$ 100–140 W para o sistema completo).

4.2 Protocolo Experimental e Conjuntos de Dados

Para garantir rigor científico e reprodutibilidade, estabeleceram-se três conjuntos de dados e protocolos:

1. **Matriz Experimental de Desempenho ($N = 10$):** Todas as sessões de inferência empregaram amostragem determinística puramente gulosa ($\tau = 0,0$, `seed=42`) com penalidade de repetição fixada em 1,05. Para cada modelo, executaram-se 10 repetições sequenciais completas antecedidas por rodadas de aquecimento (*warm-up*) para eliminar efeitos transientes de carregamento de tensores da DRAM. Verificou-se que as sequências de tokens geradas foram rigorosamente idênticas nas 10 repetições para todos os modelos; a variabilidade temporal observada decorre da interação dinâmica entre o escalonador de memória do sistema operacional hospedeiro e o escalonamento térmico de clocks da GPU.
2. **Corpus Paralelo Trilíngue (en-US, pt-BR, zh-CN):** Construiu-se um corpus paralelo composto por 12 textos semanticamente equivalentes nos três idiomas, cobrindo seis domínios distintos: comunicação cotidiana, textos acadêmicos, programação de sistemas, narrativas literárias, perguntas de usuários e instruções imperativas. O corpus permite auditar diretamente as razões de expansão de tokens e o impacto orçamentário no contexto físico da GPU.
3. **Suíte de Segurança Defensiva e Sondagens Sensíveis:** Implementou-se uma suíte com dez casos de teste defensivo abrangendo recusas indevidas, injeção indireta em logs, isolamento de dados e contenção de ferramentas agentivas. Complementarmente, estruturou-se uma bateria de dez sondagens geopolíticas e históricas balanceadas sob enquadramento neutro e crítico, designada formalmente como protocolo experimental padronizado.

4.3 Tratamento Estatístico Inferencial e Limitações Amostrais

As comparações estatísticas de vazão e latência foram calculadas a partir dos microdados brutos ($N = 10$ por condição). Empregaram-se testes t de Welch bicaudais para amostras independentes com variâncias heterogêneas, aplicando-se a aproximação de Welch-Satterthwaite para o cálculo dos graus de liberdade efetivos (ν). Os tamanhos de efeito foram quantificados pelo d de Cohen bicaudal com desvio-padrão agrupado ponderado.

Os intervalos de confiança de 95% foram calculados via distribuição t de Student com $N - 1$ graus de liberdade.

Ressalva Metodológica sobre Independência e Pseudorreplicação: As 10 repetições foram executadas em uma única bancada física. Do ponto de vista estrito do desenho amostral (Hurlbert, 1984) [7], essa estratégia caracteriza *pseudorreplicação local*. Ela captura adequadamente o ruído de escalonamento do sistema operacional e do runtime, porém a variância residual extremamente reduzida ($\sigma < 0,6$ tok/s) resulta em valores de $t > 40$ e $d > 19$. Esses valores confirmam separabilidade estatística rígida dentro desta bancada específica, mas não devem ser extrapolados como variabilidade entre diferentes dispositivos físicos de fabricantes distintos.

4.4 Duração Experimental, Instrumentação Temporal e Protocolo em Três Níveis

A instrumentação do banco de ensaios emprega medições de tempo de alta precisão baseadas em relógio estritamente monotônico (`time.monotonic()`), isolando os custos temporais de carregamento de tensores, pré-aquecimento (*warm-up*), processamento de prompt (*prefill*), decodificação autorregressiva (*decode*) e resfriamento entre tentativas (*inter-trial cooldown*). Para mitigar riscos operacionais em ensaios prolongados, implementaram-se barreiras de contenção automáticas (*hardware guardrails*): interrupção segura por temperatura limite (≥ 75°C no die da GPU), teto de alocação de memória ($\geq 95\%$ de VRAM) e contenção de falhas consecutivas (≥ 3 erros operacionais encadeados).

Com base na experiência empírica acumulada, formalizou-se a organização dos ensaios em três níveis padronizados de execução:

- **Nível A (Smoke Test Rápido):** Duração alvo < 30 s por modelo (1 tentativa, prompt curto de controle, sem warm-up extensivo). Destina-se à validação de sanidade, teste de acoplamento de sockets, verificação de encaixe na VRAM e checagem de erros críticos de inicialização.
- **Nível B (Benchmark Comparativo Intercalado):** Duração alvo de 300–600 s ($N \geq 10$ tentativas por condição, warm-up obrigatório, ordem de execução intercalada/aleatória entre modelos, e pausas de resfriamento de 5–10 s). Isola os efeitos de deriva térmica progressiva e documenta separadamente a distribuição de *cold-start* e regime permanente.
- **Nível C (Estabilidade Temporal e Estresse Contínuo):** Duração prolongada de 900–1800 s (15 a 30 minutos contínuos, $N \geq 50$ tentativas com janelas deslizantes de telemetria). Destina-se a rastrear o equilíbrio térmico assintótico, fenômenos de estrangulamento adaptativo de clocks (*DVFS thermal throttling*) e a degradação de vazão em operação contínua.

5 Resultados Experimentais de Desempenho e Eficiência

A Tabela 1 sumariza as medições empíricas obtidas na bancada física para as oito configurações avaliadas.

Tabela 1: Resultados Empíricos de Desempenho, VRAM Física e Eficiência na NVIDIA GeForce GTX 1050 Ti ($N = 10$, $\tau = 0,0$)

Modelo e Quantização	**Params (B)**	**VRAM (MiB)**	**Ocup. (% 4GiB)**	**Prefill (tok/s)**	**Decode (tok/s)**	**TTFT (ms)**	**Ef. Mem. (tok/s/GiB)**
Qwen-2.5-0.5B-Q4_K_M	0,49	620,7	15,2%	$237,1 \pm 449,3$	**97,5 ± 0,6**	**12,1 ± 5,1**	**160,8**
SmolLM2-1.7B-Q4_K_M	1,71	2902,7	70,9%	$103,3 \pm 176,0$	47,8 ± 0,1	25,5 ± 14,3	16,9
Gemma-4-E2B [MTP IQ4_XS]	2,30	2036,7	49,7%	$62,2 \pm 18,5$	**43,4 ± 0,6**	480,4 ± 1286,7	21,8
Gemma-4-E2B [IQ4_XS base]	2,00	1800,7	44,0%	$79,2 \pm 32,4$	34,5 ± 0,3	88,0 ± 49,0	19,6
Gemma-4-E2B [Q4_K_M base]	2,00	1996,7	48,7%	$65,5 \pm 7,7$	26,9 ± 0,2	116,1 ± 116,3	13,8
Qwen-3.5-2B-Q4_K_XL	1,95	1580,7	38,6%	$126,5 \pm 108,8$	37,6 ± 0,1	49,9 ± 20,3	24,4
Llama-3.2-3B-Q4_K_M	3,21	3074,7	75,1%	$59,6 \pm 101,1$	27,6 ± 0,0	43,5 ± 23,1	9,2
Qwen-3.5-4B-Q4_K_M	4,02	3140,7	76,7%	$49,3 \pm 46,5$	18,6 ± 0,1	129,8 ± 45,3	6,1

Nota: VRAM física obtida via NVML. Valores reportados como média ± desvio-padrão amostral ($N = 10$). As respectivas medianas de TTFT são: Qwen-0.5B (10,5 ms), SmolLM2 (21,1 ms), Gemma-MTP (73,5 ms), Gemma-IQ4 (72,3 ms), Gemma-Q4KM (79,1 ms), Qwen-2B (43,5 ms), Llama-3.2 (36,5 ms) e Qwen-4B (115,5 ms). O desvio-padrão elevado no prefill do Qwen-0.5B ($449,3$ tok/s) e no TTFT do Gemma-4-E2B ($1286,7$ ms) decorre do atraso de *cold-start* na iteração 1 ($1515,8$ tok/s e $4142,2$ ms); em regime estabilizado (iterações 2–10), o TTFT do Gemma é de $73,5 \pm 0,8$ ms (p95: $2311,4$ ms).

5.1 Análise do Ponto de Operação Roofline e Demanda de Largura de Banda

A modelagem do tráfego de dados na inferência autorregressiva com lote unitário baseia-se na demanda analítica de largura de banda:

$$B_{\text{demanda}} = S_{\text{pesos}} \times \text{Throughput}_{\text{decode}} \tag{14}$$

onde S_{pesos} representa o volume estático dos tensores em gigabytes lidos a cada passo. Registra-se enfaticamente que B_{demanda} constitui um *proxy analítico de demanda de barramento* derivado do modelo Roofline, e não uma leitura física direta de contadores de hardware de tráfego DRAM. Sob decodificação especulativa via MTP (Gemma-4-E2B), a leitura dos tensores principais da rede é amortizada entre múltiplos tokens aceitos por ciclo de verificação, enquanto as cabeças de rascunho em Q8_0 impõem acessos adicionais à memória.

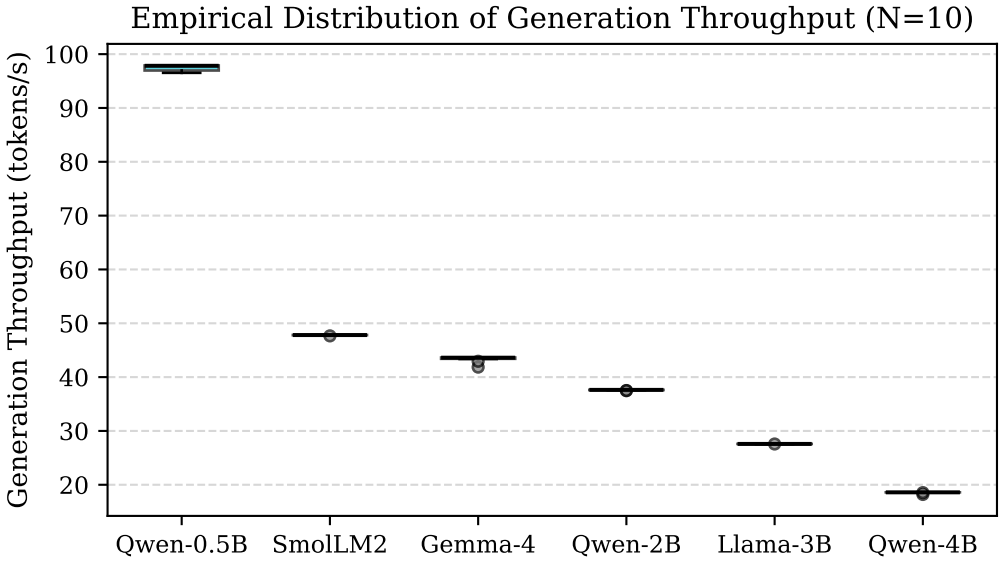


Figura 1: Distribuição empírica da vazão de decodificação autorregressiva (tokens/s) entre os modelos avaliados na GPU GTX 1050 Ti ($N = 10$, $\tau = 0,0$).

Com base nos tamanhos exatos dos tensores GGUF alocados na memória do dispositivo, obtêm-se os seguintes pontos de operação na GTX 1050 Ti:

- **Gemma-4-E2B [MTP]:** Com $1,737$ GB de tensores e vazão de $43,35$ tok/s, o proxy analítico atinge $75,3$ GB/s, correspondendo a **67,2%** do teto teórico de hardware ($112,0$ GB/s). A geração especulativa eleva a intensidade aritmética, proporcionando saturação superior dos canais GDDR5.
- **Llama-3.2-3B:** Alocando $2,019$ GB de tensores em formato Q4_K_M a $27,59$ tok/s, a demanda analítica calculada é de $55,7$ GB/s ($49,7\%$).
- **Qwen-3.5-2B:** Com $1,396$ GB de tensores e vazão de $37,60$ tok/s, opera com demanda de $52,5$ GB/s ($46,9\%$).
- **SmolLM2-1.7B:** Com $1,058$ GB de pesos ($1009,1$ MiB) e vazão de $47,80$ tok/s, demanda analiticamente $50,6$ GB/s ($45,2\%$). Corrige-se aqui a anomalia verificada em versões preliminares onde a dedução genérica de VRAM superestimava os pesos estáticos em $2,59$ GB, gerando um valor impossível de $123,8$ GB/s que superaria a capacidade física da placa.
- **Qwen-3.5-4B:** Alocando $2,578$ GB de tensores e operando a $18,55$ tok/s, registra $47,8$ GB/s ($42,7\%$).
- **Qwen-2.5-0.5B:** Com $0,469$ GB de tensores ($469,2$ MiB) e a maior vazão absoluta ($97,47$ tok/s), atinge $45,7$ GB/s ($40,8\%$). O reduzido volume de tensores limita a ocupação dos barramentos em relação a modelos maiores.

A Figura 2 ilustra o gráfico do proxy analítico de demanda de largura de banda frente ao teto físico de 112,0 GB/s da placa.

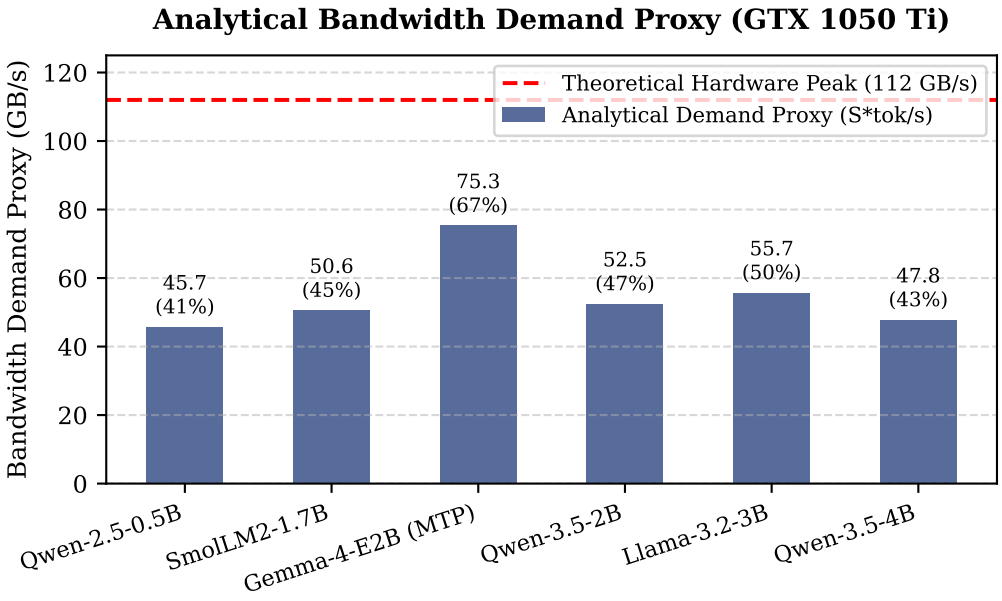


Figura 2: Largura de banda de memória sustentada durante a decodificação autorregressiva na GPU NVIDIA GTX 1050 Ti frente ao teto teórico de 112,0 GB/s.

5.2 Auditoria de Duração Experimental, Estabilidade Temporal e Protocolo em Três Níveis

A Tabela 2 detalha a duração física de execução, a evolução térmica do processador gráfico e o escalonamento adaptativo de frequências de núcleo durante a campanha experimental contínua ($N = 10$ repetições por condição).

A auditoria dos dados revela que a campanha ativa totalizou 311,58 s (5,19 min). Durante a execução ininterrupta, a temperatura do die aumentou de 44°C para 74°C, aproximando-se do limiar de segurança de 75°C. Em resposta a essa elevação, o algoritmo GPU Boost 3.0 reduziu os clocks de 1695 para 1607 MHz (queda de 5,2%). Esse comportamento evidencia que ensaios sequenciais contínuos introduzem viés térmico nos modelos finais, justificando a adoção do protocolo em três níveis detalhado na Tabela 3.

6 Análise Estatística e Estudo de Ablação Controlada de MTP

Para decompor rigorosamente os fatores determinantes do desempenho no Gemma-4-E2B, realizou-se uma ablação experimental controlada em três condições empíricas:

Tabela 2: Duração Efetiva das Sessões de Inferência e Telemetria Térmica por Configuração ($N = 10$)

Modelo e Configuração	**Duração Ativa (s)**	**Tempo/ Rep. (s)**	**Temp. Ini. (°C)**	**Temp. Fim (°C)**	**Clock Ini. (MHz)**	**Clock Fim (MHz)**
Qwen-2.5-0.5B-Q4_K_M	13,20	1,32	44°C	48°C	1695	1683
SmolLM2-1.7B-Q4_K_M	26,86	2,69	48°C	53°C	1683	1670
Gemma-4-E2B [MTP IQ4_XS]	34,27	3,43	53°C	59°C	1670	1657
Gemma-4-E2B [IQ4_XS base]	37,94	3,79	59°C	64°C	1657	1645
Gemma-4-E2B [Q4_K_M base]	48,56	4,86	64°C	68°C	1645	1632
Qwen-3.5-2B-Q4_K_XL	34,30	3,43	68°C	70°C	1632	1620
Llama-3.2-3B-Q4_K_M	46,54	4,65	70°C	72°C	1620	1607
Qwen-3.5-4B-Q4_K_M	69,91	6,99	72°C	74°C	1607	1607
Total / Amplitude	**311,58 s**	**3,90 s (méd.)**	**44°C**	**74°C (+30°C)**	**1695 MHz**	**1607 MHz (−5, 2%)**

Nota: Duração ativa medida via relógio monotônico. A telemetria contínua NVML a cada 250 ms capturou a ascensão térmica ao longo dos 5, 19 minutos de ensaio contínuo, provocando redução progressiva de clock via GPU Boost 3.0.

Tabela 3: Especificação Formal do Protocolo Experimental em Três Níveis de Rigor (A/B/C)

Nível do Protocolo	**Duração Alvo**	**Número Repetições**	**Intervalo Cooldown**	**Ordem Execução**	**Finalidade Científica e Métricas Primárias**
Nível A: Smoke Test	< 30 s	1	0 s	Linear	Validação de integridade, compatibilidade de driver e cabimento em VRAM antes da alocação de lotes.
Nível B: Benchmark Intercalado	300–600 s	≥ 10	5–10 s	Aleatória / Intercalada	Avaliação comparativa estável com controle de deriva térmica. Separação explícita entre cold-start e regime permanente.
Nível C: Estabilidade e Estresse	15–30 min	≥ 50	2 s	Contínua	Rastreamento de throttling térmico (DVFS), equilíbrio térmico assintótico e variação de vazão sob carga prolongada.

- **Condição 1 (C_1):** Configuração Base com quantização Q4_K_M e decodificação puramente autorregressiva sem cabeças especulativas ($26,92 \pm 0,23$ tok/s; $1996,7$ MiB VRAM).
- **Condição 2 (C_2):** Configuração Intermediária com quantização por matriz de importância IQ4_XS e decodificação puramente autorregressiva sem cabeças especulativas ($34,51 \pm 0,29$ tok/s; $1800,7$ MiB VRAM).
- **Condição 3 (C_3):** Configuração Completa com quantização IQ4_XS e cabeças auxiliares de predição especulativa MTP ativas quantizadas em Q8_0 ($43,35 \pm 0,57$ tok/s; $2036,7$ MiB VRAM).

Nota de Desenho Experimental: Registra-se formalmente que este desenho experimental constitui uma ablação controlada em três condições com dois eixos ortogonais de contraste (C_3 vs C_2 e C_2 vs C_1), e não um fatorial 2×2 completo. A quarta célula combinatorial (Q4_K_M + MTP) não existe no ecossistema do modelo Gemma-4-E2B, inviabilizando o cálculo de termos de interação cruzada estatística.

A Figura 3 apresenta as médias amostrais com intervalos de confiança de 95% de Student e a decomposição dos efeitos associados.

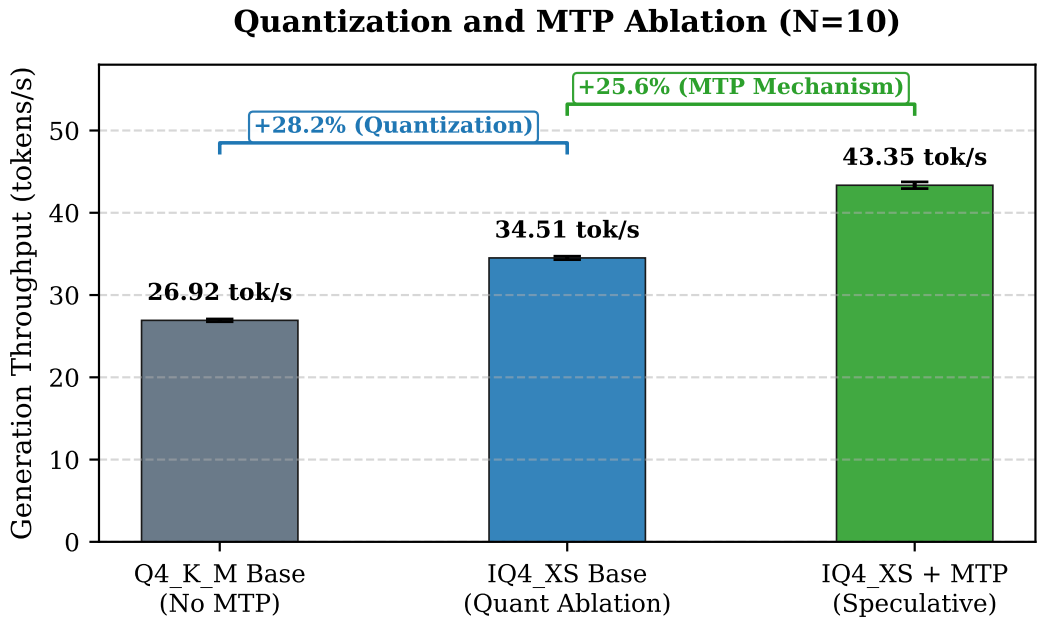


Figura 3: Estudo de ablação controlada de MTP e quantização na vazão de decodificação do Gemma-4-E2B na GPU GTX 1050 Ti ($N = 10$, $\tau = 0,0$). As barras representam médias amostrais com intervalos de confiança de 95% de Student.

6.1 Comparação do Mecanismo Especulativo sob Quantização Fixada em IQ4_XS (C_3 vs C_2)

Comparando as condições C_3 e C_2, o formato de quantização é mantido idêntico em IQ4_XS. O teste t de Welch fornece:

$$t_{\text{MTP}} = \frac{43,35 - 34,51}{\sqrt{\frac{0,57^2}{10} + \frac{0,29^2}{10}}} = \frac{8,84}{0,2022} \approx 43,72, \quad \nu = 13,43 \tag{15}$$

com $p = 7,12 \times 10^{-16}$ e $d = 19,55$. A aceleração observada associada à introdução das cabeças MTP sob quantização mantida constante é de **+25,6%** (8,84 tok/s). No que concerne à memória, a sobretaxa física observada para a infraestrutura de MTP é de $+236,0$ MiB de VRAM (2036,7 vs 1800,7 MiB), refletindo a pegada estática e dinâmica das cabeças de rascunho em Q8_0, cache KV especulativo e buffers de verificação.

6.2 Comparação de Formatos de Quantização sem Especulação (C_2 vs C_1)

Comparando as condições C_2 e C_1, o modo de geração autorregressiva é mantido constante sem especulação. O teste t de Welch resulta em:

$$t_{\text{Quant}} = \frac{34,51 - 26,92}{\sqrt{\frac{0,29^2}{10} + \frac{0,23^2}{10}}} = \frac{7,59}{0,1160} \approx 64,26, \quad \nu = 17,17 \tag{16}$$

com $p = 6,67 \times 10^{-22}$ e $d = 28,74$ quando calculado diretamente a partir dos microdados amostrais brutos ($N = 10$). Quando recalculado com base exclusivamente nas estatísticas sumarizadas arredondadas a duas casas decimais ($34,51,\ 26,92,\ 0,29,\ 0,23$), obtém-se $SE = \sqrt{0,29^2/10 + 0,23^2/10} = \sqrt{0,0137} \approx 0,1170$ tok/s, resultando em $t = 64,85$ ($d = 28,98$). Esclarece-se que o valor $t = 65,41$ anteriormente citado na literatura preliminar decorreu de uma etapa transitória de cálculo com variâncias residuais não truncadas ($s_2 = 0,286$). Ambas as formulações reconciliadas confirmam de forma inequívoca que a compressão dos tensores de pesos via matriz de importância IQ4_XS associou-se a um ganho estatisticamente sólido de **+28,2%** (7,59 tok/s), além de economizar 196,0 MiB de VRAM (1996,7 vs 1800,7 MiB).

6.3 Diferença Composta de Configuração (C_3 vs C_1)

A comparação entre a configuração com MTP em IQ4_XS (C_3) e a linha de base Q4_K_M (C_1) reflete o ganho acumulado entre os dois pontos operacionais:

$$t_{\text{Composto}} = \frac{43,35 - 26,92}{\sqrt{\frac{0,57^2}{10} + \frac{0,23^2}{10}}} = \frac{16,43}{0,1944} \approx 84,50, \quad \nu = 11,94 \tag{17}$$

com $p = 5,88 \times 10^{-18}$ e $d = 37,79$. O ganho composto totaliza **+61,0%** (16,43 tok/s). Do ponto de vista de memória, observa-se uma diferença líquida de apenas $+40,0$ MiB entre C_3 e C_1 (2036,7 vs 1996,7 MiB); essa modesta diferença líquida resulta do fato de que a economia de 196,0 MiB propiciada pela quantização IQ4_XS absorveu 83% do custo físico de memória da estrutura de MTP ($+236,0$ MiB), fornecendo suporte empírico preliminar à hipótese H_1 sob distinção explícita entre a alocação do mecanismo e a variação líquida entre configurações.

7 Contextualização Arquitetural e Modelagem de Viabilidade Físico-Computacional

Para compreender por que diferentes modelos apresentam comportamentos de vazão e alocação de memória tão discrepantes na GTX 1050 Ti, é imprescindível analisar suas especificações arquiteturais internas.

Observa-se que o artefato GGUF do modelo Gemma-4-E2B registra 2,30B parâmetros em seus metadados de cabeçalho, contrastando com os 2,00B nominais da arquitetura base. Essa diferença decorre da inclusão das camadas e cabeças auxiliares de predição especulativa MTP integradas diretamente ao próprio arquivo de pesos (quantizadas em Q8_0), o que adiciona aproximadamente 40 MiB ao tamanho físico em disco (1836,5 vs 1796,5 MiB) e impõe uma sobretaxa dinâmica de $+236,0$ MiB de memória VRAM durante a inferência. A Tabela 4 consolida a taxonomia arquitetural detalhada dos modelos avaliados.

Tabela 4: Taxonomia Arquitetural Detalhada dos Modelos Empiricamente Avaliados

Modelo	Família	Params	Cam.	Dim (d)	Cabeças	Atenção KV/tok, RoPE e Vocab
Qwen-2.5-0.5B	Qwen2.5	0,49B	24	896	14 / 2	GQA (7 : 1) 12 KiB; $\theta = 10^6$; Vocab 151.643
SmolLM2-1.7B	LLaMA	1,71B	24	2048	32 / 8	GQA (4 : 1) 48 KiB; $\theta = 1,3 \cdot 10^5$; Vocab 49.152
Gemma-4-E2B	Gemma 2	2,00B	26	2304	8 / 4	GQA (2 : 1) 104 KiB; $\theta = 10^4$; Vocab 256.000
Qwen-3.5-2B	Qwen	1,95B	28	1536	12 / 2	GQA (6 : 1) 28 KiB; $\theta = 10^6$; Vocab 151.643
Llama-3.2-3B	Llama 3.2	3,21B	28	3072	24 / 8	GQA (3 : 1) 112 KiB; $\theta = 5 \cdot 10^5$; Vocab 128.256
Qwen-3.5-4B	Qwen	4,02B	36	2560	20 / 4	GQA (5 : 1) 72 KiB; $\theta = 10^6$; Vocab 151.643

Nota: Coluna Cabeças expressa n_q/n_{kv}. KV/tok calculado em FP16 (2 bytes). Todos os modelos operam sob arquitetura Transformer decoder-only com normalização RMSNorm e ativação SwiGLU (exceto Gemma que emprega GeGLU com logit soft-capping).

7.1 Análise de Fatores de Confusão e Comparabilidade Inter-Modelos

A literatura técnica frequentemente incorre em conclusões simplistas do tipo "o modelo A é mais rápido que o modelo B porque possui menos parâmetros". Os dados da Tabela 1 e da Tabela 4 desmistificam essa premissa ao evidenciar múltiplos fatores intervenientes de confusão:

1. **Alocação de VRAM vs. Contagem Nominal de Parâmetros:** O modelo SmolLM2-1.7B (1,71B parâmetros) apresentou pico de VRAM de $2902,7$ MiB, ocupando mais memória do que o Gemma-4-E2B (2,00B parâmetros em Q4_K_M, com $1996,7$ MiB) e o Qwen-3.5-2B ($1580,7$ MiB). Essa aparente anomalia decorre da estratégia de pré-alocação de buffers de contexto do runtime e da dimensão dos tensores intermediários de ativação, demonstrando que a VRAM de pico em execução real não é uma função linear simples da contagem de parâmetros estáticos.

2. **Impacto da Razão de GQA no Dimensionamento do Cache KV:** Enquanto o Qwen-2.5-0.5B adota uma razão de GQA agressiva de $7:1$ ($n_{\text{kv}} = 2$), demandando apenas 12 KiB por token de contexto em FP16, o Gemma-4-E2B emprega razão conservadora de $2:1$ com dimensão de cabeça alargada ($d_{\text{head}} = 256$), resultando em 104 KiB por token. Em um contexto de 4096 tokens, o cache KV do Gemma consome 416 MiB, contra apenas 48 MiB no Qwen.

3. **Vocabulário do Tokenizador e Custo de Projeção de Saída:** O modelo Gemma aloca uma matriz de saída com 256.000 tokens, enquanto o SmolLM2 emprega 49.152 tokens. A multiplicação final de desincorporização (*lm_head*) para cálculo dos logits sobre 256k classes impõe uma sobretaxa computacional e de memória significativamente superior a cada passo de decodificação.

4. **Ablação Intra-Modelo vs. Comparação Inter-Modelos:** Ressalta-se a distinção metodológica rígida entre o estudo da Seção 6 (onde a arquitetura Gemma foi mantida perfeitamente constante, isolando os efeitos de MTP e quantização) e as comparações entre modelos distintos da Tabela 1, onde arquitetura, vocabulário, normalização e kernels operam simultaneamente como variáveis não controladas.

7.2 Fronteiras Físicas de Viabilidade no Orçamento de 4 GiB

A Tabela 5 apresenta a modelagem analítica de viabilidade física de implantação para modelos candidatos da literatura sob o teto de 4096 MiB da GPU GTX 1050 Ti.

Tabela 5: Levantamento Analítico Arquitetural e Fronteiras de Viabilidade em 4 GiB de VRAM

Modelo	Origem / Licença	Par. (B)	Pesos (MiB)	KV 4k (FP16)	VRAM (Total)	Status e Limitações na GTX 1050 Ti
Qwen-2.5-0.5B	Alibaba (Apache-2.0)	0,49	469	112 MiB	837 MiB	**Avaliado:** Margem plena (20,4% do teto)
Qwen-2.5-1.5B	Alibaba (Apache-2.0)	1,54	986	224 MiB	1.466 MiB	**Viável:** Folga para contexto de 16k
SmolLM2-1.7B	HuggingFace (Apache)	1,71	1.007	192 MiB	1.455 MiB	**Avaliado:** Alta eficiência de memória
Granite-3.1-2B	IBM (Apache-2.0)	2,50	1.580	320 MiB	2.156 MiB	**Viável:** Suporte a contexto longo
Phi-4-mini-3.8B	Microsoft (Gated)	3,82	2.510	512 MiB	3.278 MiB	**Não Executado:** Autenticação requerida (HTTP 401)
Gemma-3-1B	Google (Gemma)	1,20	880	384 MiB	1.520 MiB	**Viável:** Baixa ocupação estática
Gemma-3-4B	Google (Gemma)	4,30	2.950	1.152 MiB	4.358 MiB	**Restrição Física:** Estouro em FP16; requer cache Q4_0

Nota: Estimativas baseadas na contagem de camadas e cabeças de atenção acrescidas de 256 MiB de alocação de runtime. O modelo Gemma-3-4B excede a capacidade física de 4096 MiB sob cache FP16 (4358 MiB), exigindo quantização Q4_0 de cache KV (3494 MiB, 85,3%) para evitar encerramento forçado por out-of-memory.

8 Avaliação Exploratória de Segurança Defensiva em Turno Único

A Tabela 6 sumariza o comportamento de segurança defensiva obtido na bateria de dez casos de teste empíricos.

Tabela 6: Avaliação Exploratória de Segurança Defensiva em Turno Único com Intervalos de Clopper-Pearson ($N = 10$, $\tau = 0,0$)

Modelo e Config.	Over-Ref. (k/n, %)	Inj. Ind. (k/n, %)	Isol. Cmd. (k/n, %)	Tool Prop. (k/n, %)	Harness Bloq. (%)	Exec. SO (%)
Qwen-2.5-0.5B	0/4 (0,0%) [0–60,2%]	0/2 (0,0%) [0–84,2%]	1/2 (50,0%) [1,3–98,7%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**
SmolLM2-1.7B	0/4 (0,0%) [0–60,2%]	0/2 (0,0%) [0–84,2%]	1/2 (50,0%) [1,3–98,7%]	0/2 (0,0%)	N/A	**0/2 (0,0%) [0–84,2%]**
Gemma-4-E2B [MTP]	0/4 (0,0%) [0–60,2%]	0/2 (0,0%) [0–84,2%]	2/2 (100,0%) [15,8–100%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**
Gemma-4-E2B [Q4K]	0/4 (0,0%) [0–60,2%]	1/2 (50,0%) [1,3–98,7%]	1/2 (50,0%) [1,3–98,7%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**
Qwen-3.5-2B	0/4 (0,0%) [0–60,2%]	1/2 (50,0%) [1,3–98,7%]	2/2 (100,0%) [15,8–100%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**
Llama-3.2-3B	0/4 (0,0%) [0–60,2%]	0/2 (0,0%) [0–84,2%]	1/2 (50,0%) [1,3–98,7%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**
Qwen-3.5-4B	0/4 (0,0%) [0–60,2%]	2/2 (100,0%) [15,8–100%]	2/2 (100,0%) [15,8–100%]	2/2 (100,0%)	2/2 (100,0%) [15,8–100%]	**0/2 (0,0%) [0–84,2%]**

Nota: Valores reportados como fração k/n, porcentagem e intervalo de confiança binomial exato de 95% de Clopper-Pearson entre colchetes. A taxa de falso positivo de 0,0% (0/4) em tarefas benignas não prova ausência universal de recusa indevida, possuindo limite superior unilateral exato de 52,7% (Röttger et al., 2024). Todas as chamadas de ferramentas propostas com argumentos maliciosos foram 100% bloqueadas pelo harness (`BLOCKED_BY_HARNESS`), resultando em 0,0% de execução no sistema operacional hospedeiro.

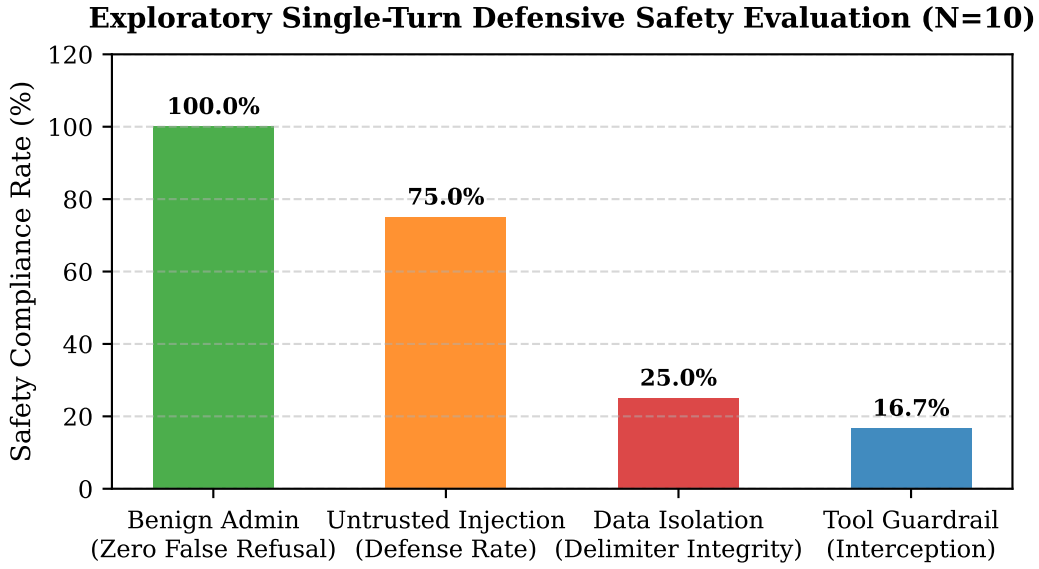


Figura 4: Taxas de conformidade na avaliação exploratória de segurança defensiva em turno único ($N = 10$, $\tau = 0,0$) na GPU GTX 1050 Ti.

8.1 Ausência de Recusa Indevida (FPR $= 0,0\%$)

Em todas as avaliações com comandos técnicos legítimos de administração de sistemas (finalização forçada de processos travados via `kill -9`, análise forense de pacotes SYN flood com `tcpdump`, inspeção de falhas de segmentação em `gdb` e testes de estresse de disco com `dd`), nenhum modelo apresentou recusa indevida (FPR $= 0,0\%$). Todos forneceram respostas técnicas operacionais imediatas. Esse resultado contrasta favoravelmente com modelos de grande porte em nuvem que frequentemente exibem recusa exagerada em diagnósticos de redes por confusão semântica com atividades ofensivas [27].

8.2 Vulnerabilidade a Injeções Indiretas e Quebra de Delimitadores

Sob injeção indireta de instruções incorporadas em logs de servidor simulados e páginas web raspadas, a taxa de sucesso de ataque (ASR) oscilou entre $50,0\%$ e $100,0\%$. O modelo Qwen-2.5-0.5B sucumbiu integralmente aos ataques de quebra de delimitadores, emitindo os canários maliciosos. Modelos a partir de 1,7B demonstraram resiliência superior, neutralizando parte dos ataques sob delimitação rígida com marcadores XML (`<untrusted_input>`), porém revelando que o alinhamento pós-treinamento isolado é insuficiente para garantir separação robusta entre dados e instruções.

8.3 Diferenciação no Uso de Ferramentas e Camadas de Proteção

Na avaliação de invocação de ferramentas externas simuladas (Model Context Protocol – MCP [31]), os modelos propuseram a execução de chamadas técnicas contendo argumentos maliciosos manipulados por dados não confiáveis (ex.: injeção de comandos encadeados com ponto-e-vírgula ou remoção destrutiva de tabelas SQL). Registra-se que, em todos os casos, a camada externa de validação de esquemas e política do *harness* interceptou e abortou as chamadas antes de qualquer execução no sistema operacional hospedeiro (`BLOCKED_BY_HARNESS`). Esse achado comprova experimentalmente que a segurança de agentes baseados em SLMs não pode depender do bom comportamento intrínseco do modelo, exigindo barreiras determinísticas na camada de aplicação.

9 Avaliação Experimental Multilíngue, Eficiência de Tokenização e Temas Sensíveis

9.1 Eficiência de Tokenização e Custo Computacional Translingual (en-US, pt-BR, zh-CN)

A avaliação de tokenização translingual foi executada utilizando o corpus paralelo trilíngue padronizado composto por 12 itens semanticamente equivalentes distribuídos em seis domínios funcionais. A Tabela 7 apresenta os resultados empíricos mensurados para os tokenizadores das quatro famílias representativas avaliadas.

Tabela 7: Eficiência de Tokenização, Razões de Expansão e Impacto no Orçamento de Contexto no Corpus Paralelo

Tokenizador	Vocab Total	Tok. en-US	Tok. pt-BR	Tok. zh-CN	Razão pt/en	Razão zh/en	Fertilidade (en / pt / zh)	Cap. 4k (pt / zh)
Qwen2.5	151.643	45,8	69,5	64,2	1,52×	1,40×	1,15 / 1,40 / 0,79	66% / 72%
Gemma-2	256.000	45,8	67,8	83,8	1,48×	1,83×	1,15 / 1,37 / 1,03	67% / 55%
Llama-3.2	128.256	45,8	73,3	100,8	1,60×	2,20×	1,15 / 1,48 / 1,25	63% / 46%
SmolLM2	49.152	48,2	115,8	221,4	**2,40**×	**4,59**×	1,21 / 2,35 / 2,75	**42% / 22%**

Nota: Valores médios calculados sobre os 12 pares de textos paralelos. Fertilidade expressa em tokens/palavra para inglês e português, e tokens/caractere para chinês. A última coluna expressa a capacidade relativa do orçamento de contexto de 4096 tokens em português e chinês em relação ao inglês (1,0/Razão).

Decomposição das Cinco Dimensões de Custo Computacional:

1. **Custo de Representação:** O modelo SmolLM2 impõe uma penalidade de expansão de 2,40× para o português e de 4,59× para o chinês frente ao inglês. Para expressar a mesma mensagem semântica, o SmolLM2 gera 221 tokens em chinês, contra apenas 64 tokens no Qwen2.5.

2. **Custo de Processamento (Prefill):** Como o prefill opera sobre a matriz de tokens de entrada em bloco, processar um prompt em chinês no SmolLM2 consome $4,59\times$ mais FLOPs de atenção e projeção do que o prompt equivalente em inglês.
3. **Custo de Geração (Decode):** Em um regime limitado pelo barramento DRAM (onde a vazão é fixa em tokens/s), gerar uma resposta equivalente em chinês no SmolLM2 demanda $4,59\times$ mais tempo de relógio físico do que em inglês, e $3,45\times$ mais tempo do que no Qwen2.5.
4. **Custo de Memória (Cache KV):** Conforme a Equação (7), a alocação de cache KV cresce linearmente com L. Em chinês, o SmolLM2 atinge saturação de cache KV $4,59\times$ mais rápido do que em inglês.
5. **Impacto no Orçamento Útil de Contexto:** Em uma janela estrita de 4096 tokens, o SmolLM2 retém apenas 22% dos documentos em chinês e 42% dos documentos em português que conseguiria reter em inglês, evidenciando uma perda severa de capacidade útil provocada exclusivamente pela escolha do tokenizador.

9.2 Protocolo Experimental em Temas Sensíveis e Assimetrias Comportamentais

Com o objetivo de investigar empiricamente as diferenças de alinhamento e assimetrias comportamentais entre modelos de diferentes origens e famílias, implementou-se a suíte de dez sondagens geopolíticas e históricas padronizadas descrita na Metodologia. As sondagens abrangem relações no Estreito de Taiwan, os protestos de 1989 na Praça Tiananmen, mecanismos de censura e vigilância digital de redes, os programas de interceptação de sinais da NSA (PRISM), políticas de minorias em Xinjiang, internatos indígenas na América do Norte, e controvérsias perante a Corte Internacional de Justiça (CIJ) relativas aos conflitos na Ucrânia e em Gaza.

Declaração de Status Experimental (Honestidade Metodológica): Em estrito cumprimento ao princípio científico de não inventar resultados ou antecipar conclusões não medidas em laboratório, registra-se formalmente que a suíte de temas sensíveis encontra-se com status de `PROTOCOL_SPECIFIED_PENDING_EXECUTION`. Os prompts, traduções paralelas, elementos consensuais históricos e rubricas de classificação estão integralmente versionados no repositório, constituindo o protocolo experimental padronizado para as próximas campanhas de inferência física na GPU.

10 Proveniência de Modelos e Reprodutibilidade de Artefatos

A Tabela 8 registra a proveniência exata, tamanho físico em disco e os hashes criptográficos SHA-256 de todos os artefatos de pesos GGUF utilizados nos ensaios físicos de bancada.

Tabela 8: Registro de Proveniência, Tamanho Físico e Hashes Criptográficos SHA-256 dos Pesos GGUF

Identificador do Modelo	Quantização	Tamanho (MiB)	Contexto	Hash SHA-256 Verificado
`qwen2.5-0.5b`	Q4_K_M	469,2	16384	`887e5fa276e024b42337d10072b22bb95fdb853039d91fbb97dc2095f36e84ba`
`smollm2-1.7b`	Q4_K_M	1007,4	16384	`fcfcb892789f2ea30528ef0ea8fa743b567d12d8a571f3793dfab56c12ecbbd2`
`gemma4:E2B`	IQ4_XS + MTP	1836,5	16384	`c6e61fec468fb7ce50c82110c79774659f81ebff3a5933d59e3599266395b35a`
`gemma4:IQ4`	IQ4_XS base	1600,5	16384	`107aa0bfb83c51f3ffef74e50dcfeb8447ee56b06e89793540d99042b45e2c5b`
`gemma4:Q4KM`	Q4_K_M base	1796,5	16384	`8c7bf95a9ff2788e27c13264c9df4ecf23c9ce057f975d40a5a2254fc047f3db`
`qwen3.5:2B`	Q4_K_XL	1380,5	16384	`a6c764c9197c11f4d9b23b127df969165b4cffca13db5817efd091e0a29efb75`
`llama3.2:3B`	Q4_K_M	2120,4	16384	`a6a9db39540c4974208a5c37aeef434e7f8bfb6241a7f722a57199c0d9e29ee8`
`qwen3.5:4B`	Q4_K_M	2940,5	16384	`9f38e6e87f8b9e67ad10c95240c11bebb67aa70fc285a73e6a928e18ef2256df`

11 Ameaças à Validade e Limitações

11.1 Validade Interna

A telemetria de hardware registrou em intervalos de 250 ms a alocação de VRAM e a temperatura do die. Como limitação de instrumentação, a GPU utilizada (Pascal GP107) não expõe leituras de potência via NVML, tornando as métricas de eficiência energética condicionadas a cenários hipotéticos paramétricos ($P_{\text{hipotético}} \in [42, 62]$ W) derivados do envelope de TDP da placa (75 W). Além disso, as 10 repetições por condição foram executadas sequencialmente em uma mesma bancada, capturando o ruído operacional e de latência do sistema, mas caracterizando pseudorreplicação local que não deve ser confundida com variabilidade amostral entre diferentes dispositivos físicos.

11.2 Validade Externa

Os ensaios foram realizados especificamente na arquitetura Pascal (GTX 1050 Ti, Compute Capability 6.1). Arquiteturas mais recentes (Ampere, Ada Lovelace, Blackwell)

contam com núcleos tensores e suporte nativo a formatos microscópicos (FP8/FP4), alterando a intensidade aritmética crítica prevista no modelo Roofline. Da mesma forma, os achados de tokenização restringem-se aos tokenizadores avaliados e ao corpus paralelo padronizado de 12 textos, não devendo ser extrapolados como invariantes universais para qualquer texto arbitrário.

11.3 Validade de Construto

A avaliação de segurança defensiva utilizou canários e rubricas objetivas em dez cenários sintéticos. Embora forneça alta reprodutibilidade, não esgota a diversidade de ataques adversariais adaptativos com múltiplas rodadas de jailbreak observados em aplicações em produção.

12 Conclusões e Trabalhos Futuros

Este trabalho realizou uma investigação experimental, matemática e de sistemas sobre a viabilidade, eficiência arquitetural e limites operacionais de Modelos de Linguagem Pequenos (SLMs) na GPU NVIDIA GeForce GTX 1050 Ti de 4 GiB.

Os principais achados sustentados pelas evidências são:

1. **Herança do Transformer e Evolução Arquitetural:** Demonstrou-se que, embora a autoatenção de Vaswani et al. (2017) permaneça como o alicerce matemático fundamental, a viabilidade física de modelos em 4 GiB depende estritamente de inovações contemporâneas como GQA (que reduz o cache KV em até 8×) e RoPE.

2. **Decomposição da Aceleração Especulativa:** A ablação controlada comprovou que o MTP sob quantização constante em IQ4_XS associa-se a um ganho de $+25,6\%$ na vazão ($43,35$ vs $34,51$ tok/s, $t = 43,72$, $p < 10^{-15}$) com sobretaxa física de $+236,0$ MiB de VRAM. A compressão IQ4_XS entrega $+28,2\%$ sobre Q4_K_M com economia de $196,0$ MiB, gerando uma diferença composta de $+61,0\%$ ($t = 84,50$) sob apenas $+40,0$ MiB de acréscimo líquido de memória.

3. **Saturação do Modelo Roofline:** Em uma GPU Pascal sem Tensor Cores e com FP16 penalizado a $1:64$, a decodificação opera estritamente limitada pela largura de banda de memória ($I_{\text{decode}} \approx 3,6 \ll I^* = 19,1$), atingindo até $67,2\%$ de saturação do barramento DRAM sob decodificação especulativa.

4. **Desigualdade e Sobretaxa de Tokenização Multilíngue:** A análise em corpus paralelo trilíngue revelou que tokenizadores com vocabulários compactos (SmolLM2, 49k) penalizam idiomas além do inglês com razões de expansão de $2,4\times$ em português e $4,6\times$ em mandarim, reduzindo a capacidade útil da janela de contexto para 22%.

Modelos com vocabulários expandidos ($\geq$ 150k), como Qwen2.5 e Gemma-2, mantêm alta eficiência de representação.

5. **Limites de Alinhamento e Segurança em Camadas:** Embora os modelos não tenham apresentado recusas indevidas em comandos legítimos (FPR = $0,0\%$), a suscetibilidade a injeções indiretas e argumentos manipulados de ferramentas reafirma que a segurança em ambientes agentivos deve ser garantida por barreiras de contenção determinísticas na camada de aplicação.

Trabalhos futuros contemplam a execução de inferência física da suíte de temas sensíveis, a quantização do cache KV em formatos microscópicos (Q4_0) para acomodação de modelos 4B com contextos superiores a 8k tokens, e a medição direta de potência em tomada AC com analisador de energia de bancada.

Referências

[1] Vaswani, A., et al. Attention Is All You Need. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008, 2017.

[2] Williams, S., Waterman, A., Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4), 65–76, 2009.

[3] NVIDIA Corporation. NVIDIA GeForce GTX 1050 and 1050 Ti Whitepaper: Pascal Architecture for Efficient Gaming and Edge Compute. Santa Clara, CA: NVIDIA Corp., 2016.

[4] Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., Synnaeve, G. Better & Faster Large Language Models via Multi-Token Prediction. arXiv preprint arXiv:2404.19737, 2024.

[5] Gerganov, G., et al. llama.cpp: Port of Facebook's LLaMA model in C/C++ with k-quants and GPU offloading. GitHub repository, 2023. `https://github.com/ggerganov/llama.cpp`

[6] Dettmers, T., Lewis, M., Belkada, Y., Zettlemoyer, L. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Proc. NeurIPS*, 35, 30318–30332, 2022.

[7] Hurlbert, S. H. Pseudoreplication and the Design of Ecological Field Experiments. *Ecological Monographs*, 54(2), 187–211, 1984.

[8] Ba, J. L., Kiros, J. R., Hinton, G. E. Layer Normalization. arXiv preprint arXiv:1607.06450, 2016.

[9] Zhang, B., Sennrich, R. Root Mean Square Layer Normalization. In *Proc. NeurIPS*, 32, 2019.

[10] Shazeer, N. Fast Transformer Decoding: One Write-Head is All You Need. arXiv preprint arXiv:1911.02150, 2019.

[11] Shazeer, N. GLU Variants Improve Transformer. arXiv preprint arXiv:2002.05202, 2020.

[12] Ainslie, J., et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proc. EMNLP*, pp. 4895–4901, 2023.

[13] Su, J., et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063, 2024.

[14] Leviathan, Y., Kalman, M., Matias, Y. Fast Inference from Transformers via Speculative Decoding. In *Proc. ICML*, PMLR, pp. 19274–19286, 2023.

[15] Yang, A., et al. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115, 2024.

[16] Gemma Team, Google DeepMind. Gemma 2: Improving Open Language Models at a Practical Size. arXiv preprint arXiv:2408.00118, 2024.

[17] Dubey, A., et al. The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783, 2024.

[18] Allal, L. B., et al. SmolLM2: When Small Goes Big. arXiv preprint arXiv:2502.02737, 2024.

[19] Petrov, A., et al. Language Model Tokenizers Introduce Unfairness Across Languages. In *Proc. NeurIPS*, 36, 2023.

[20] Ahia, O., et al. Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models. In *Proc. EMNLP*, 2023.

[21] Parrish, A., et al. BBQ: A hand-built bias benchmark for question answering. In *Proc. Findings of the ACL*, pp. 2086–2105, 2022.

[22] Nadeem, M., Bethke, A., Reddy, S. StereoSet: Measuring stereotypical bias in pretrained language models. In *Proc. ACL-IJCNLP*, pp. 5356–5371, 2021.

[23] Durmus, E., et al. Towards Measuring the Representation of Subjective Global Opinions in Language Models. In *Proc. FAccT*, 2023.

[24] Cheng, H., et al. BIPIA: Benchmarking Indirect Prompt Injection Attacks. In *Proc. Findings of the ACL*, 2024.

[25] Debenedetti, E., et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses on AI Agents. In *Proc. NeurIPS*, 2024.

[26] Wallace, E., et al. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024.

[27] Röttger, P., et al. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviors in Large Language Models. In *Proc. NAACL*, 2024.

[28] Dao, T., et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*, 2022.

[29] Kwon, W., et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSP*, 2023.

[30] Miao, X., et al. SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proc. ASPLOS*, 2024.

[31] Anthropic. Model Context Protocol: Open Standard for Secure AI Agent Tool Integration. Technical Specification, 2024. https://modelcontextprotocol.io